



Automate your DR solution for relational databases on AWS

AWS Prescriptive Guidance



AWS Prescriptive Guidance: Automate your DR solution for relational databases on AWS

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Introduction	1
Objectives	1
DR Orchestrator Framework overview	3
The failover process	3
DR Orchestrator FAILOVER architecture	4
The failback process	5
DR Orchestrator FAILBACK architecture	5
DR Orchestrator FAILOVER	7
Parameter details	8
DR Orchestrator FAILBACK	9
Aurora states and Step Functions state machines	12
Steady state	12
Event state	12
Switchover	13
Manual unplanned failover	18
Failback	22
Deploy the solution	31
Prerequisites	31
Limitations	31
Deploy example databases on AWS	31
FAQ	33
What are the RPO and RTO I can achieve using this approach?	33
Is it mandatory to use AWS CloudFormation export variables?	33
Can I use the DR Orchestrator FAILOVER workflow to fail over more than one AWS database?	33
How can I avoid the InvalidParameterCombination error when I run the DR Orchestrator FAILBACK state machine for Amazon RDS?	34
Next steps	35
Resources	36
Document history	37

Automate your DR solution for relational databases on AWS

Jitendra Kumar, Pavithra Balasubramanian, and Oliver Francis, Amazon Web Services

When planning a [disaster recovery \(DR\) strategy](#) on Amazon Web Services (AWS), you can implement a DR solution for the databases in your organization. You can automate the DR solution by configuring an event-driven architecture to perform the failover of the database instances. The benefits of this approach include the following:

- Reducing human errors caused by manual coordination
- Providing the shortest possible recovery time objective (RTO) in some cases
- Facilitating repeatable testing of your DR solution

Without automation, an outage in your primary AWS Region, would require resources in your organization to manually perform steps in a run-book. Those steps might include multiple processes, such as promoting Amazon Relational Database Service (Amazon RDS) read replicas in the secondary AWS Region to serve as the new primary database instance. AWS provides [AWS SDK for Python \(Boto3\)](#) APIs to automate these actions, which can form the building blocks of a DR solution for your organization's needs.

This guide discusses a centralized, manifest-driven DR orchestrator which can help you achieve a DR solution for AWS databases across AWS Regions within the same AWS account. The decision to perform a failover would still need human intervention and is out of scope for the automation.

The target audiences for this guide are application architects, infrastructure architects, database architects and database consultants.

Objectives

Implementing the DR Orchestrator Framework solution can help you achieve the following outcomes:

- **Reduce manual errors during an outage** – Manual performance of DR processes can be prone to errors. Automating actions during the phases of a DR activity reduces the manual errors and helps to ensure a smoother failover or fallback.

- **Manage DR at scale** – If your organization has a large fleet of databases, building automation for DR helps you scale your DR solution. You can automate your AWS database DR solution by using DR Orchestrator Framework. DR Orchestrator automates the steps for failover or failback of AWS databases by using a parameter file in JSON format.

DR Orchestrator Framework overview

DR Orchestrator Framework provides a one-click solution to orchestrate and automate cross-Region DR for AWS databases. It uses [AWS Step Functions](#) and [AWS Lambda](#) to perform the required steps during the failover and failback. The Step Functions state machines provide the basis for decision making within the orchestrator design. The API operations for performing a failover or failback actions are coded into Lambda functions that are called from within the state machine. The Lambda functions run [AWS SDK for Python \(Boto3\)](#) APIs to interact with AWS databases.

DR Orchestrator Framework contains two main state machines that correspond to the failover and failback phases.

For Amazon RDS, the failover phase promotes a cross-Region RDS read replica into a standalone DB instance. For Amazon Aurora, when the primary Region is down during a rare, unexpected outage, its writer node isn't available. Replication between the writer node and the secondary clusters stops. You must detach the secondary cluster from the global database and promote it as a standalone cluster. Applications can connect and send write traffic to the standalone cluster. You can use this same process to [switch over](#) the primary DB cluster of the global database to the secondary Regions. Use this approach for controlled scenarios such as the following:

- Operational maintenance
- Planned operational procedures
- Promotion of an Amazon ElastiCache (Redis OSS) secondary cluster as your new primary cluster

The failback phase establishes live replication of data between a live primary Region and a new secondary Region.

It's critical to understand that DR Orchestrator applies to databases only. All the applications that reference these databases and are in the same Region might need a separate, tandem failover solution. After the databases fail over to the secondary Region, the applications need to be updated to connect to the new database instances, which will serve as the data source.

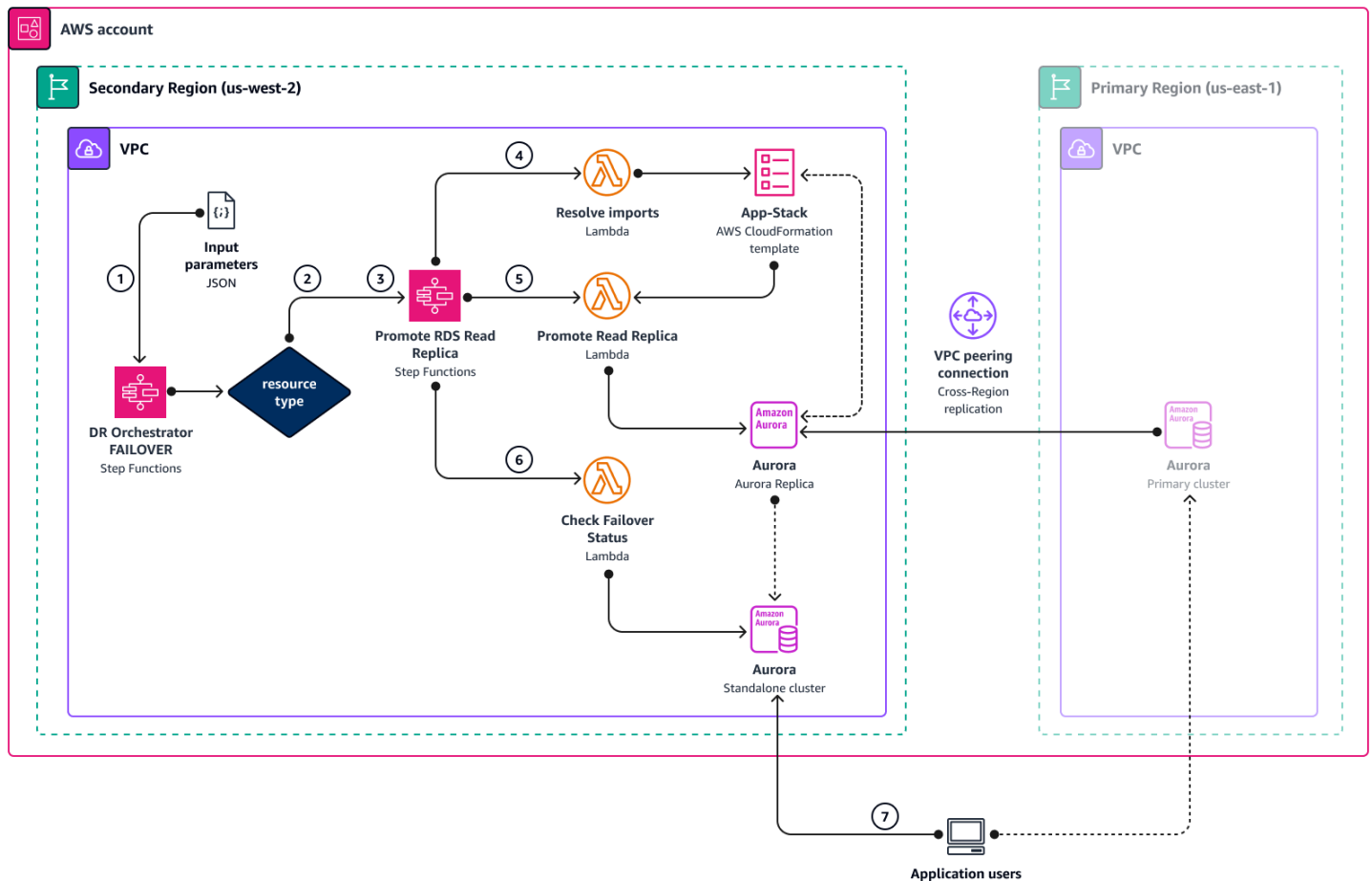
The failover process

To perform a failover, run the `DR Orchestrator FAILOVER` state machine. At this stage, a secondary database is already present in the secondary Region, either as a read replica (Amazon

RDS) or as a secondary cluster (Amazon Aurora). When you run the DR Orchestrator FAILOVER state machine, it promotes the secondary database to become the primary.

DR Orchestrator FAILOVER architecture

The following diagram shows the concepts of the failover process for Amazon Aurora when using DR Orchestrator. Amazon Aurora and Amazon ElastiCache use the same workflow but with different state machines and Lambda functions.



1. The DR Orchestrator FAILOVER state machine reads the input JSON parameters.
2. Based on the resourceType parameter, the state machine calls other state machines: Promote RDS Read Replica, Failover Aurora Cluster, or Failover ElastiCache. If more than one resource is passed in the input, these state machines run in parallel.
3. The Failover Aurora Cluster state machine calls Lambda functions in each of the following three steps.
4. The Resolve imports Lambda function resolves "! import <export-variable-name>" with the actual values from the App-Stack AWS CloudFormation template.

5. The `Failover Aurora Cluster Lambda` function promotes the read replica as a standalone DB instance.
6. The `Check Failover Status Lambda` function checks the status of the promoted DB instance. After the status is **AVAILABLE**, the Lambda function sends a success token back to the calling state machine and completes.
7. You can redirect your applications to the standalone database in the DR Region (`us-west-2`), which is now the primary database.

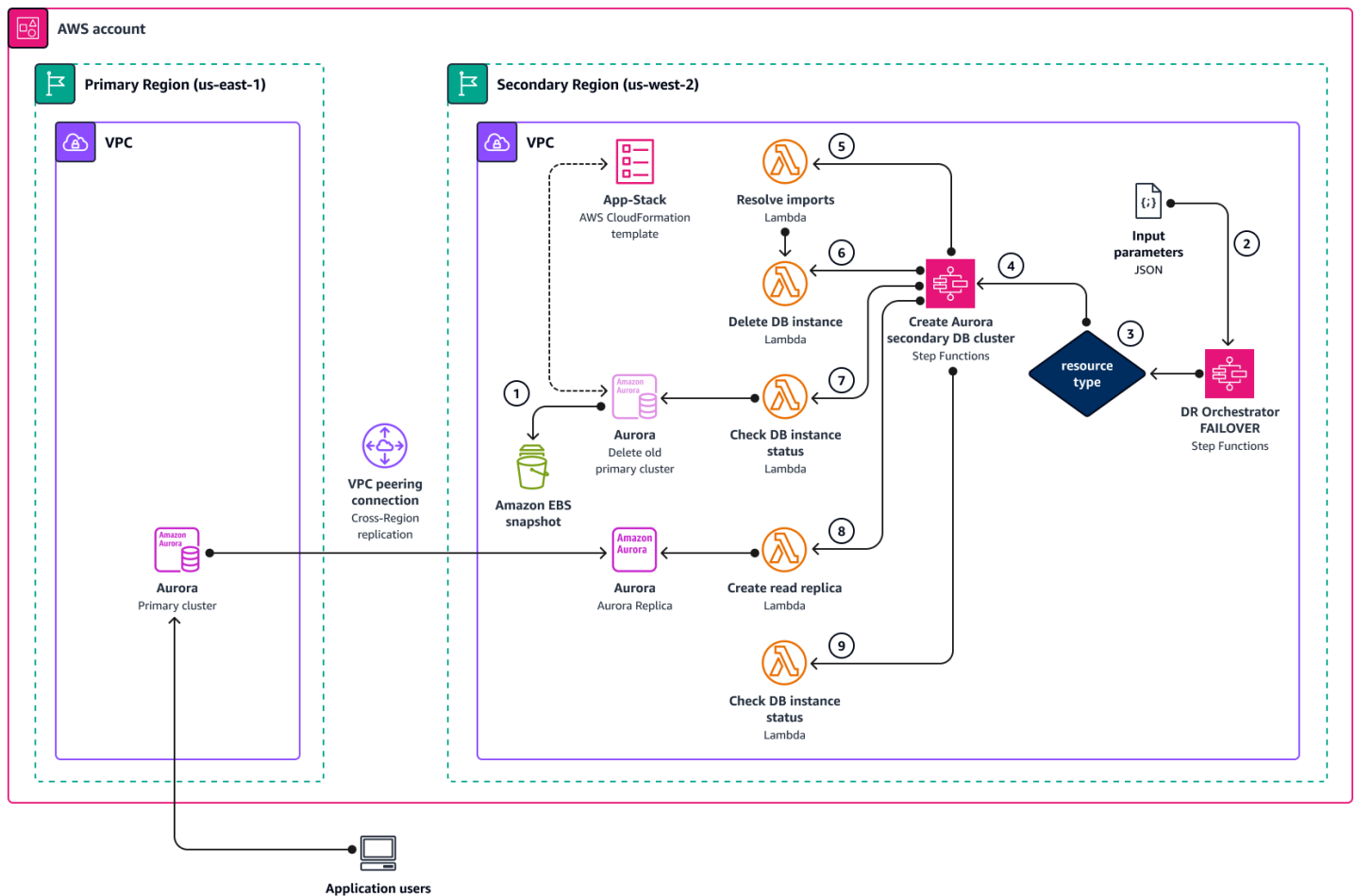
The failback process

After your former primary Region (`us-east-1`) is up again, you can fail back to it, so that the database in `us-east-1` becomes the primary again. To start the failback, run the `DR Orchestrator FAILBACK` state machine. As the name indicates, this state machine starts replicating changes in your new primary Region (`us-west-2`) back to the former primary Region (`us-east-1`), which acts as the current secondary.

After replication is established between the two Regions, you can initiate the failback. To failback and return to your original primary Region (`us-east-1`), run the `DR Orchestrator FAILOVER` state machine in the current secondary Region (`us-east-1`) to promote it to the primary Region.

DR Orchestrator FAILBACK architecture

The following diagram shows the concepts of the failback process for Amazon Aurora when using DR Orchestrator.



1. Before beginning failback, take a manual DB snapshot to use when performing root cause analysis (RCA).

Also, disable the DeletionProtection for the Aurora cluster in the previous primary Region (us-east-1).

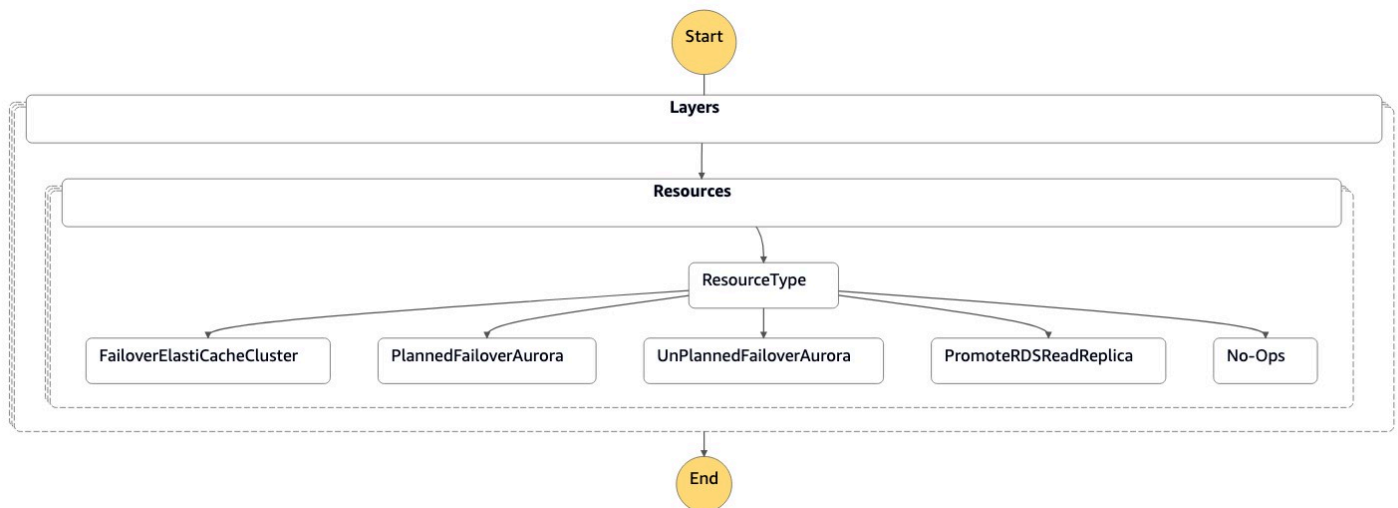
2. The DR Orchestrator FAILBACK state machine reads the input JSON parameters.
3. Based on the resourceType, the DR Orchestrator FAILBACK state machine calls the Create Aurora Secondary DB Cluster state machine.
4. The Create Aurora Secondary DB Cluster state machine calls Lambda functions in each of the following five steps.
5. The Resolve import Lambda function resolves `! import <export-variable-name>` with the actual values from the App-Stack CloudFormation template.
6. The Delete DB Instance Lambda function deletes the former primary instance.

7. The Check DB instance status Lambda function checks the Delete DB Instance status until the DB is deleted.
8. The Create Read Replica Lambda function creates a read replica in the secondary Region from the DB instance that's in the new primary Region.
9. The Check DB instance status Lambda function checks the read replica DB instance status. When the status is **AVAILABLE**, the Lambda function sends a success token back to the calling state machine, which is completed.

DR Orchestrator FAILOVER

Use the DR Orchestrator FAILOVER state machine in the DR event when the primary Region (us-east-1) is down or during planned events such as operational maintenance.

The function can be called to fail over single or multiple databases in parallel.



The state machine accepts parameters in the JSON format as shown in the following code:

```

{
  "StatePayload": [
    {
      "layer": 1,
      "resources": [
        {
          "resourceType": "PromoteRDSReadReplica",
          "resourceName": "Promote RDS MySQL Read Replica",
          "parameters": {

```

```

        "RDSInstanceIdentifier": "!Import rds-mysql-instance-identifier",
        "TargetClusterIdentifier": "!Import rds-mysql-instance-global-arn"
    },
    {
        "resourceType": "FailoverElastiCacheCluster",
        "resourceName": "Failover ElastiCache Cluster",
        "parameters": {
            "GlobalReplicationGroupId": "!Import demo-redis-cluster-global-replication-
group-id",
            "TargetRegion": "!Import demo-redis-cluster-target-region",
            "TargetReplicationGroupId": "!Import demo-redis-cluster-target-replication-
group-id"
        }
    }
]
}
}

```

Parameter details

The following table shows the parameters used by the DR Orchestrator FAILOVER state machine.

Parameter name	Description	Expected values
layer (required: number)	The processing sequence. All the resources defined in layer 1 must be run before the layer 2 resources are run.	1 or 2, and so on
resources (required: array of dictionary)	All the resources within a single layer run in parallel.	

```

{
  "resourceType": "
String",
  "resourceName": "St
ring",
  "parameters": {
    "<param1>": "<!Import
cft-output-1">,
    ....
  }
}

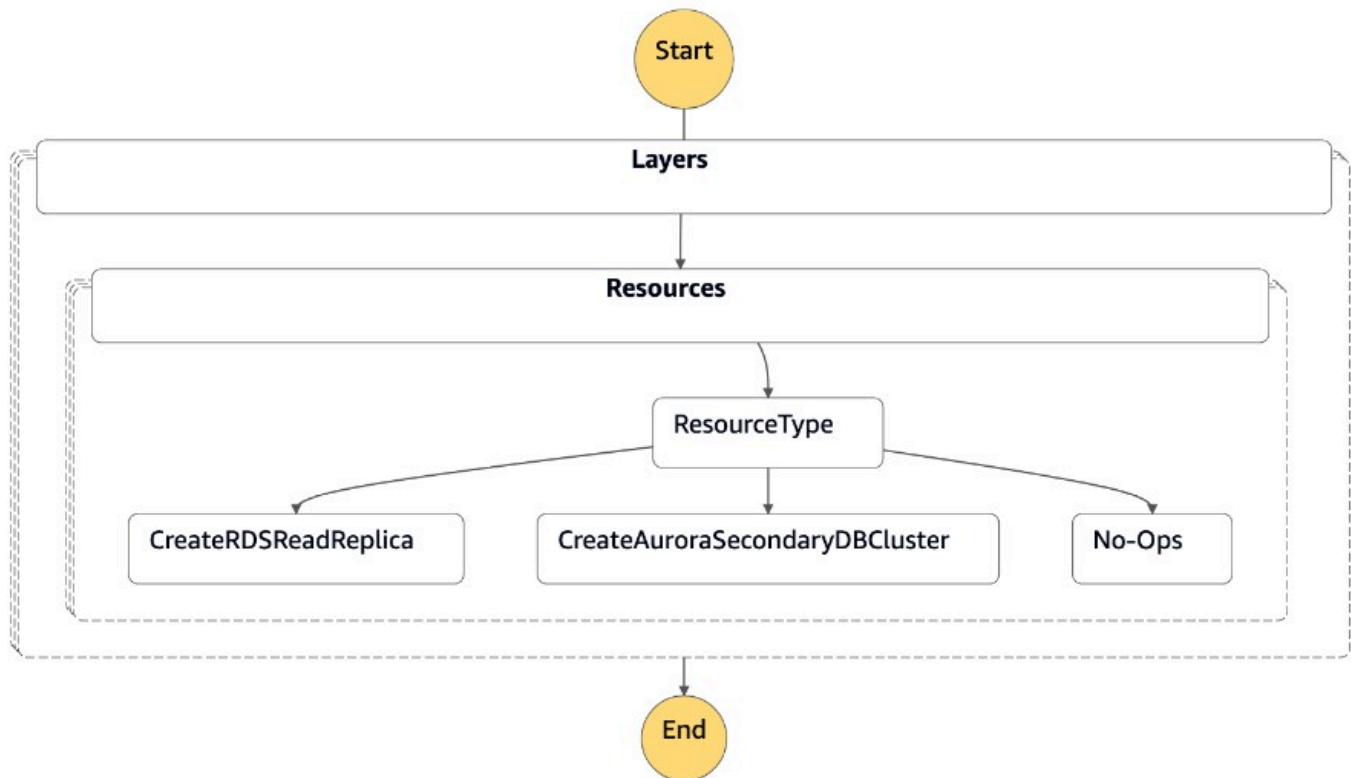
```

}

<code>resourceType</code> (required: string)	Type of the resource to identify the resource	PromoteRDSReadReplica or FailoverElastiCacheCluster
<code>resourceName</code> (optional: string)	To identify which application portfolio these resources belong to	Promote RDS for MySQL Read Replica
<code>parameters</code> (required: array of dictionary)	List of parameters required to fail over or fail back the AWS database	<pre>{ "<param1>": "<!Import cft-output-1>", "<param2>": "<!Import cft-output-2>", }</pre>

DR Orchestrator FAILBACK

Use the DR Orchestrator FAILBACK state machine after the DR event, when the former primary Region (us-east-1) is up. You can create the [read replica](#) for Amazon RDS in the former primary Region from the new primary Region (us-west-2) to be compliant with your DR strategy. Because this is a planned event, you can schedule this activity over the weekend or during off-peak business hours with an estimated downtime.



The state machine accepts parameters in the JSON format as shown in the following code:

```

{
  "StatePayload": [
    {
      "layer": 1,
      "resources": [
        {
          "resourceType": "CreateRDSReadReplica",
          "resourceName": "Create RDS for MySQL Read Replica",
          "parameters": {
            "RDSInstanceIdentifier": "!Import rds-mysql-instance-identifier",
            "TargetClusterIdentifier": "!Import rds-mysql-instance-global-arn",
            "SourceRDSInstanceIdentifier": "!Import rds-mysql-instance-source-identifier",
            "SourceRegion": "!Import rds-mysql-instance-SourceRegion",
            "MultiAZ": "!Import rds-mysql-instance-MultiAZ",
            "DBInstanceClass": "!Import rds-mysql-instance-DBInstanceClass",
            "DBSubnetGroup": "!Import rds-mysql-instance-DBSubnetGroup",
            "DBSecurityGroup": "!Import rds-mysql-instance-DBSecurityGroup",
            "KmsKeyId": "!Import rds-mysql-instance-KmsKeyId",
            "BackupRetentionPeriod": "7",

```

```
        "MonitoringInterval": "60",
        "StorageEncrypted": "True",
        "EnableIAMDatabaseAuthentication": "True",
        "DeletionProtection": "True",
        "CopyTagsToSnapshot": "True",
        "AutoMinorVersionUpgrade": "True",
        "Port": "!Import rds-mysql-instance-DBPortNumber",
        "MonitoringRoleArn": "!Import rds-mysql-instance-RDSMonitoringRole"
    }
}
]
```

Aurora states and Step Functions state machines

This section covers the process and state machines specific to failing over and failing back Amazon Aurora clusters. The clusters are configured as a global database.

Note

For the demonstration purposes, this example uses Aurora MySQL-Compatible Edition. You can use similar steps for Aurora PostgreSQL-Compatible Edition.

Steady state

In the steady state, an Amazon Aurora MySQL-Compatible global database (`dr-globaldb-cluster-mysql`) has been created with two DB clusters. The first DB cluster (`db-cluster-01`) has been created in the primary AWS Region (`us-east-1`) to serve the read/write workload. The second DB cluster (`db-cluster-02`) has been created in the secondary Region (`us-west-2`) to server the read-only workload.

In addition to providing the DR solution, you can reduce the load on your primary DB cluster by routing read queries from your applications to the secondary DB cluster. Each of these clusters contains one database instance called `dbcluster-01-use1-instance-1` and `dbcluster-02-usw2-instance-2`, respectively.

Event state

By using an Amazon Aurora global database, you can plan for and recover from disaster fairly quickly. Recovery from disaster is typically measured using values for recovery time objective (RTO) and recovery point objective (RPO). For more information, see [Using switchover or failover in an Amazon Aurora global database](#).

With an Aurora global database, there are two different approaches to failover:

- Switchover (managed planned failover)
- Failover (manual unplanned failover, or *detach and promote*)

Switchover

Switchover is intended for controlled environments, such as operational maintenance and other planned operational procedures. By using a managed planned failover, you can relocate the primary DB cluster of your Aurora global database to one of the secondary Regions. Because switchover waits until the secondary DB clusters are synchronized with the primary database, RPO is 0 (no data loss). To learn more, see [Performing switchovers for Amazon Aurora global databases](#).

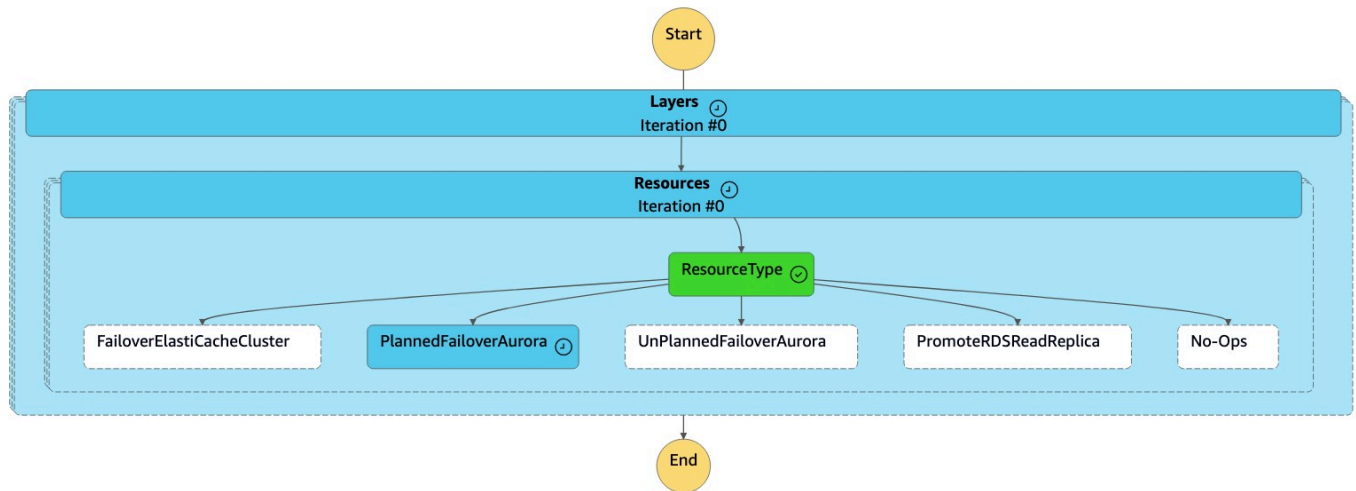
The `dr-orchestrator-stepfunction-FAILOVER` state machine is invoked during the *event state* to switch your primary cluster over to your chosen secondary Region (`us-west-2`).

To perform the switchover, do the following:

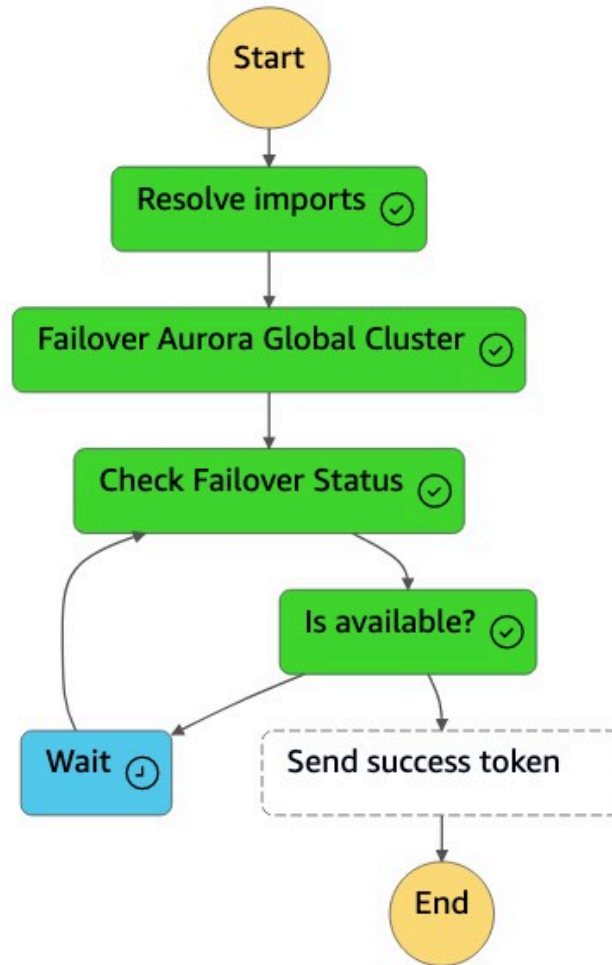
1. Sign in to the AWS Management Console.
2. Change the Region to the DR Region (`us-west-2`).
3. Navigate to **Services**, and choose **Step Functions**.
4. Navigate to the `dr-orchestrator-stepfunction-FAILOVER` state machine.
5. Choose **Start execution**, and enter the following JSON code in the Input - optional section:

```
{
  "StatePayload": [
    {
      "layer": 1,
      "resources": [
        {
          "resourceType": "PlannedFailoverAurora",
          "resourceName": "Switchover (planned failover) of Amazon Aurora global databases (MySQL)",
          "parameters": {
            "GlobalClusterIdentifier": "!Import dr-globaldb-cluster-mysql-global-identifier",
            "DBClusterIdentifier": "!Import dr-globaldb-cluster-mysql-cluster-identifier"
          }
        }
      ]
    }
  ]
}
```

6. The `dr-orchestrator-stepfunction-FAILOVER` state machine reads the resource type as `PlannedFailoverAuroraMySQL`, and it calls the `dr-orchestrator-stepfunction-planned-Aurora-failover` state machine to fail over the Aurora global database.



7. The `dr-orchestrator-stepfunction-planned-Aurora-failover` state machine performs the following steps to switch over the Aurora MySQL-Compatible global database role.



Step	Description	Expected Values
Resolve imports	A Lambda function replaces <code>!Import <variable name></code> values with the actual name.	<code>!Import dr-global db-cluster-mysql-global-identifier</code> is replaced by <code>"dr-global db-cluster-mysql"</code> .

Failover Aurora Global Cluster

A Lambda function calls [failover_global_cluster](#) Boto3 APIs to fail over the Aurora global database.

```
{
  'GlobalCluster': {
    'GlobalClusterId': 'dr-global-cluster-mysql',
    'GlobalClusterIdentifier': 'dr-global-cluster-mysql',
    'GlobalClusterResourceId': 'cluster-cce7f9bec2846db4',
    'GlobalClusterArn': 'arn:aws:rds::xxx',
    'Status': 'failing-over',
    ....
  }
}
```

Check Failover Status

A Lambda function calls [describe_db_clusters](#) Boto3 APIs to check the status of the failover.

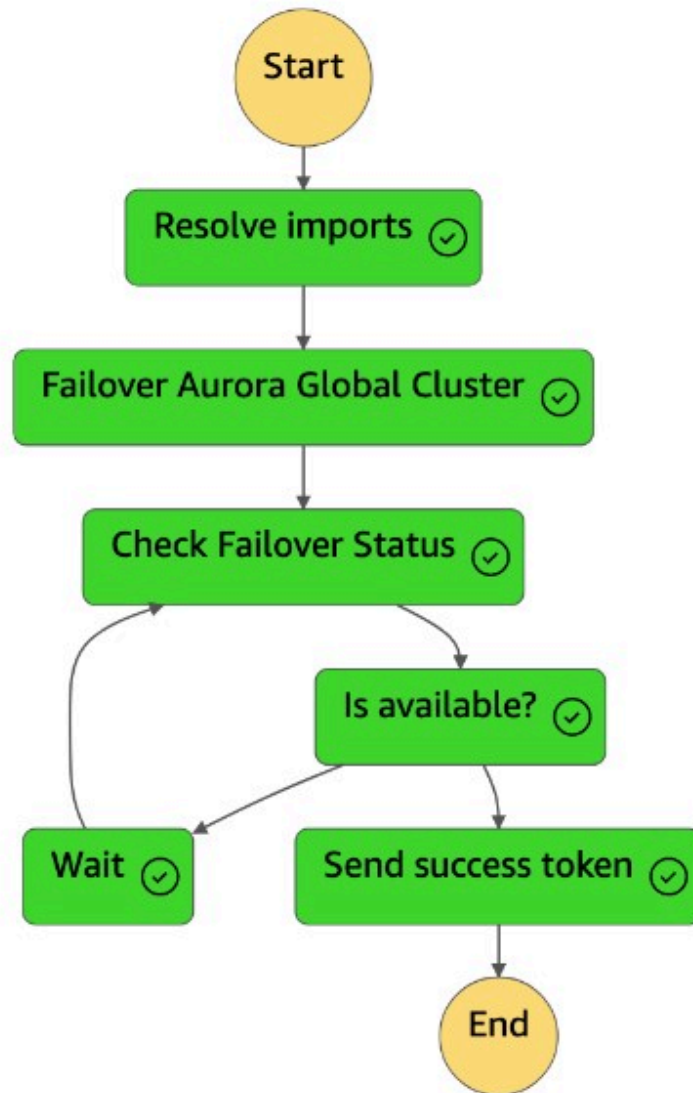
modifying, available

Send success tokens

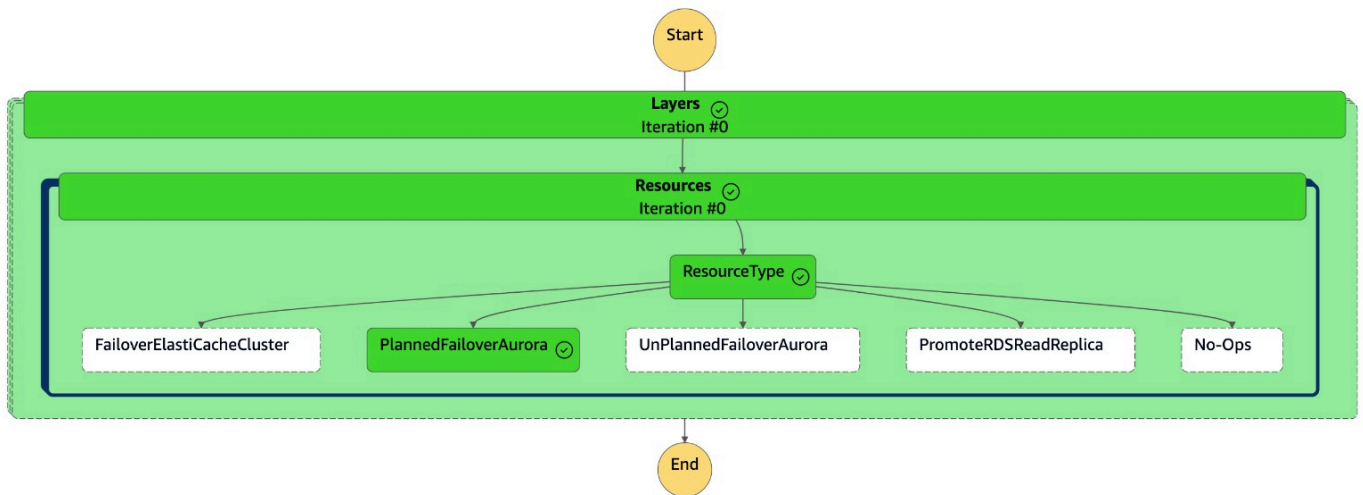
A Lambda function calls [send_task_success](#) Boto3 APIs and sends a success token back to the DR Orchestrator Failover state machine.

h7XRIcdLtd/83p1E0d
MccoxlzFhglSDKzPK9
mBVKZsp7d9yrT1W

8. Navigate to the Amazon RDS console. Under **Status**, the values for the Aurora global database will change from **Available** to **Switching over** or **Modifying**.
9. After the `dr-orchestrator-stepfunction-planned-Aurora-failover` state machine is completed, it sends a success token back to the `dr-orchestrator-stepfunction-FAILOVER` state machine.



10.The `dr-orchestrator-stepfunction-FAILOVER` state machine is completed.



On the console, the role of the **Secondary cluster** (dbcluster-02) is now **Primary cluster**, and the cluster is ready to serve read/write workloads. The role of the original primary cluster (dbcluster-01) is now listed as **Secondary cluster**.

Manual unplanned failover

On rare occasions, your Aurora global database might experience an unexpected outage in its primary AWS Region. If this happens, your primary Aurora DB cluster and its writer node aren't available, and the replication between the primary cluster and the secondaries ceases. To minimize both downtime (RTO) and data loss (RPO), work quickly to perform a cross-Region failover and reconstruct your Aurora global database. For more information, see [Recovering an Amazon Aurora global database from an unplanned outage](#).

Performing an unplanned failover requires you to detach your secondary cluster from the Aurora global database. Before you perform the unplanned failover, stop application writes on your primary Aurora DB cluster. After the failover is completed successfully, reconfigure the application to write to the new primary DB cluster. This approach helps prevent data loss. It also helps avoid data inconsistencies if the primary writer node comes back online during the failover process.

To perform the unplanned failover, call the `dr-orchestrator-stepfunction-FAILOVER` state machine. For this example, the **Secondary cluster** (db-cluster-02) is in the DR Region (us-west-2) in the steady state.

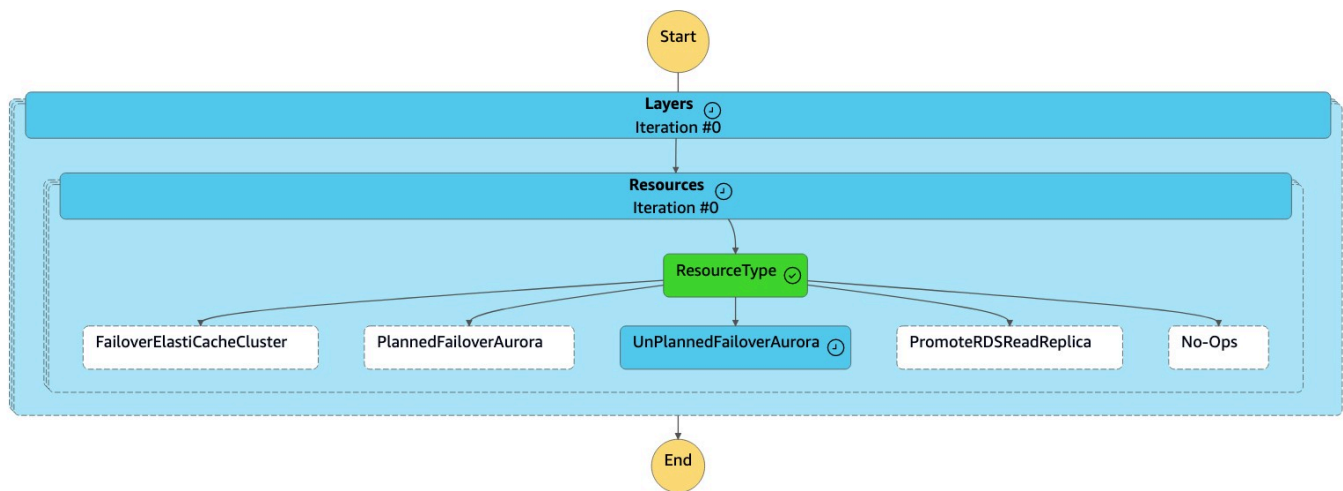
To perform the failover, do the following:

1. Sign in to the console.

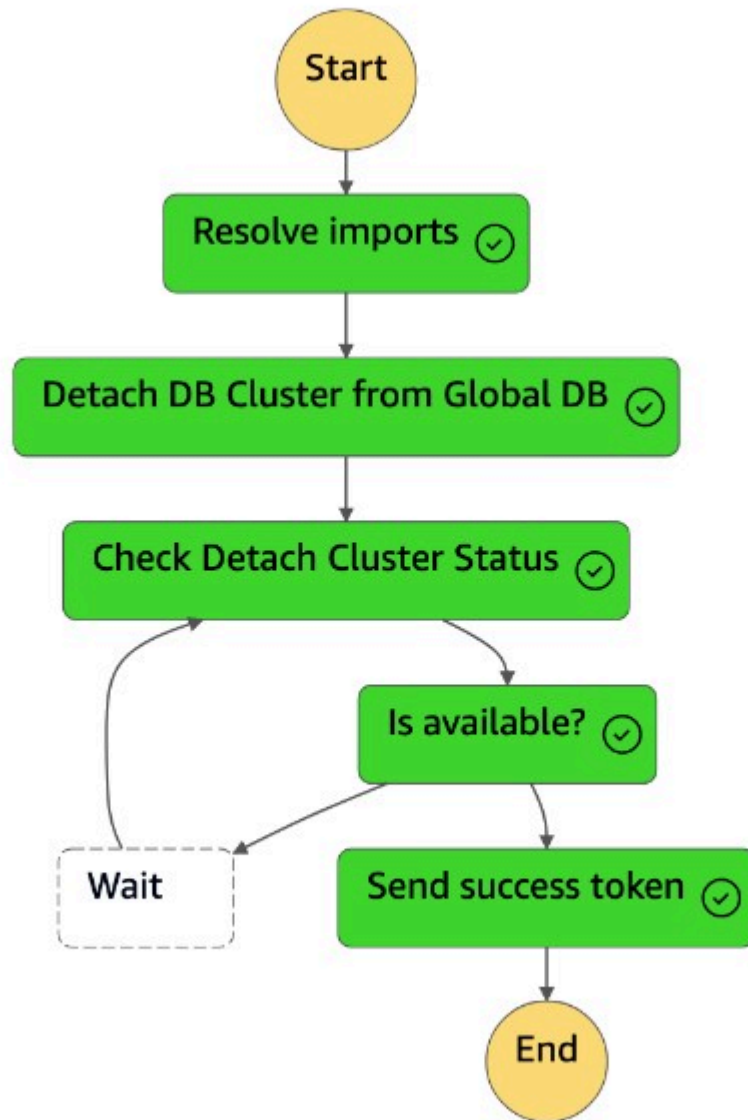
2. Change the Region to the DR Region (us-west-2).
3. Navigate to **Services**, and choose **Step Functions**.
4. Navigate to the dr-orchestrator-stepfunction-FAILOVER state machine.
5. Choose **Start execution**, and enter the following JSON code in the Input - optional section, using UnPlannedFailoverAurora as the resourceType:

```
{
  "StatePayload": [
    {
      "layer": 1,
      "resources": [
        {
          "resourceType": "UnPlannedFailoverAurora",
          "resourceName": "Performing unplanned failover for Amazon Aurora global
databases (MySQL)",
          "parameters": {
            "GlobalClusterIdentifier": "!Import dr-globaldb-cluster-mysql-global-
identifier",
            "DBClusterIdentifier": "!Import dr-globaldb-cluster-mysql-cluster-
identifier",
            "ClusterRegion": "!Import dr-globaldb-cluster-mysql-cluster-region"
          }
        }
      ]
    }
  ]
}
```

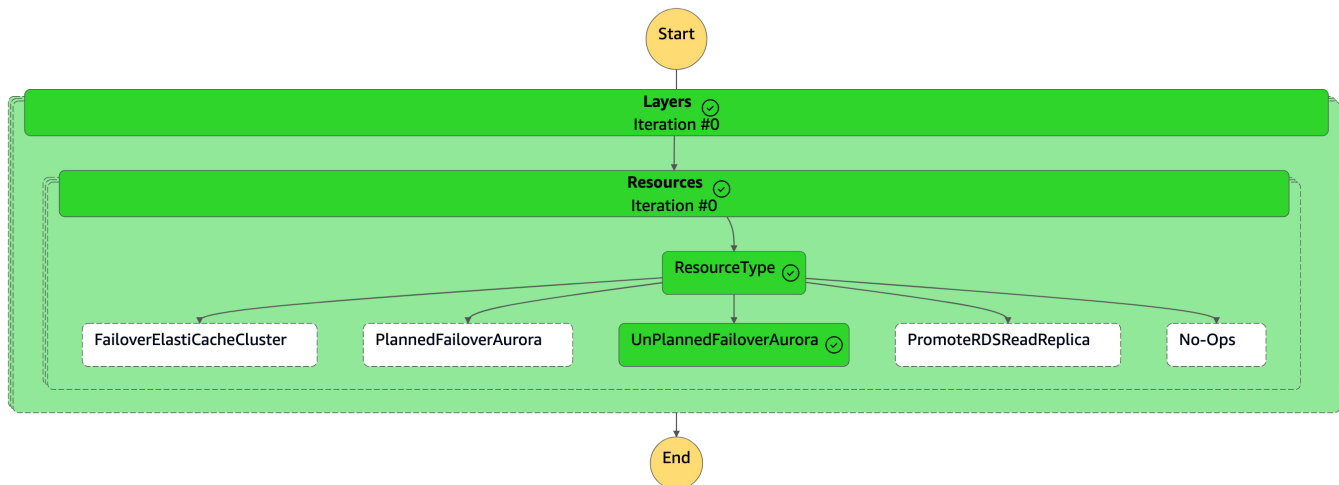
6. The dr-orchestrator-stepfunction-FAILOVER state machine reads the resource type as UnPlannedFailoverAuroraMySQL and calls the task Detach Cluster from Global Database from the dr-orchestrator-stepfunction-unplanned-Aurora-failover state machine.



7. The `Detach Cluster from Global Database` task detaches (removes) the secondary cluster from the global database.



8. The secondary cluster (dbcluster-02) is promoted to become a standalone cluster, and it can serve read/write workloads.
9. The `dr-orchestrator-stepfunction-FAILOVER` state machine is completed.



10. The secondary cluster (dbcluster-02) is detached from the Aurora global database, and it becomes a standalone cluster to serve the read/writer workload.

11. Reconfigure your application to send all write operations to this new standalone Aurora DB cluster by using its new cluster endpoint.

Failback

A failback returns your database to the original (or new) primary location after a disaster (or a scheduled event) is resolved. When the unplanned outage has been resolved, you might want to add your former primary Region back to the Aurora global database. You must first delete the existing DB cluster from the former primary Region, create a new DB cluster from the new primary Region, and then use the managed planned failover process to switch over the new cluster's role.

This can be considered as a planned activity that you can perform during off-peak hours or on a weekend.

You must manually [modify the Amazon Aurora DB Cluster](#) and disable the DeletionProtection before you run the DR Orchestrator FAILBACK state machine from the former primary Region (us-east-1) because it was created with DeletionProtection.

DR Orchestrator Framework uses the `dr-orchestrator-stepfunction-FAILBACK` state machine to automate the steps to delete the existing cluster and create a new cluster in the former primary Region.

To disable DeletionProtection, do the following:

1. Sign in to the console.
2. Change the Region to the former primary Region (us-east-1).
3. Navigate to the Amazon RDS console, select the cluster name (dbcluster-01), and choose **Modify**.
4. Under **Deletion protection**, clear the **Enable deletion protection** check box, and choose **Continue**.
5. Choose **Apply immediately**, and then choose **Modify cluster**.

The DR Orchestrator FAILBACK state machine is invoked during the failback process from the former primary Region (us-east-1).

To perform the failback, do the following:

1. Sign in to the console.
2. Change the Region to the former primary Region (us-east-1).
3. Navigate to **Services**, and then choose **Step Functions**.
4. Navigate to the DR Orchestrator FAILBACK state machine.
5. Choose **Start execution**, and enter the following JSON code in the Input - optional section:

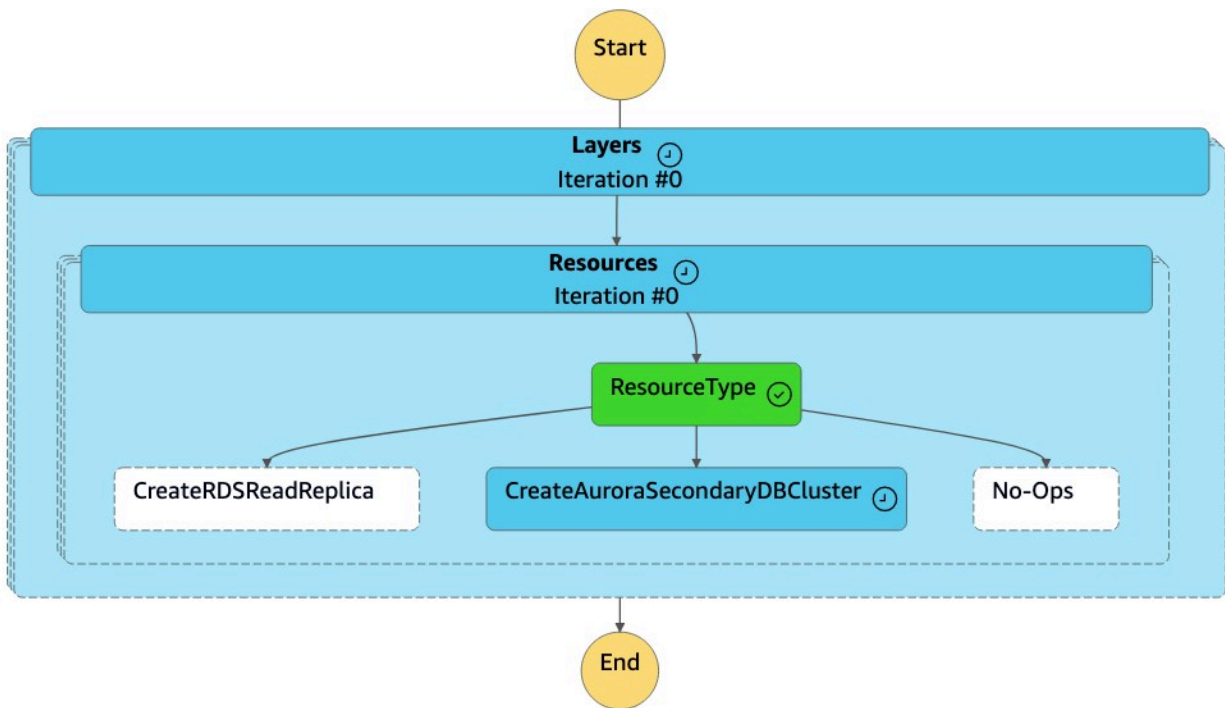
```
{
  "StatePayload": [
    {
      "layer": 1,
      "resources": [
        {
          "resourceType": "CreateAuroraSecondaryDBCluster",
          "resourceName": "To create secondary Aurora MySQL Global Database Cluster",
          "parameters": {
            "GlobalClusterIdentifier": "!Import dr-globaldb-cluster-mysql-global-identifier",
            "DBClusterIdentifier": "!Import dr-globaldb-cluster-mysql-cluster-identifier",
            "DBClusterName": "!Import dr-globaldb-cluster-mysql-cluster-name",
            "SourceDBClusterIdentifier": "!Import dr-globaldb-cluster-mysql-source-cluster-identifier",
            "DBInstanceIdentifier": "!Import dr-globaldb-cluster-mysql-instance-identifier",
            "Port": "!Import dr-globaldb-cluster-mysql-port",
```

```

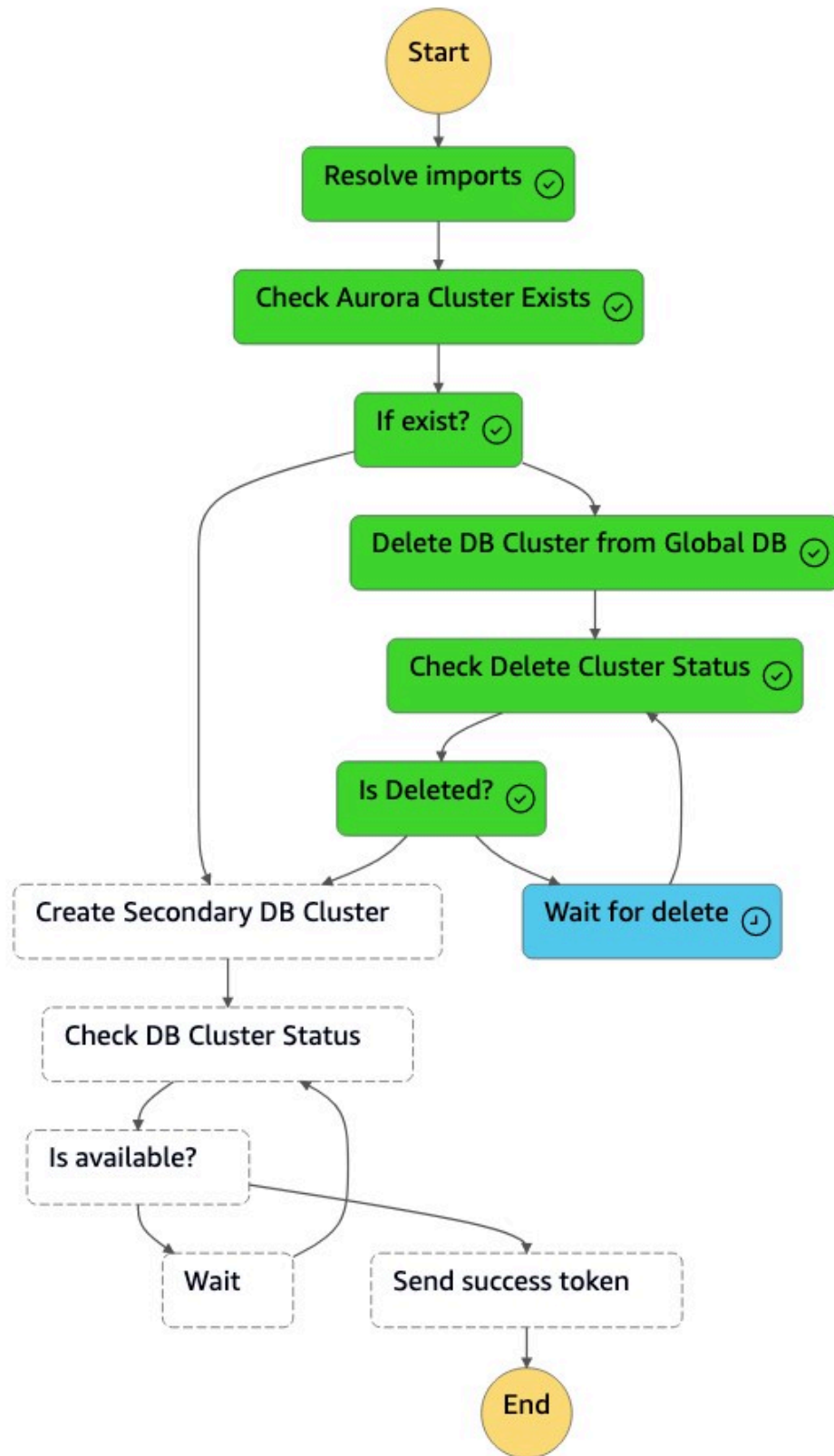
        "DBInstanceClass": "!Import dr-globaldb-cluster-mysql-instance-class",
        "DBSubnetGroupName": "!Import dr-globaldb-cluster-mysql-subnet-group-
name",
        "VpcSecurityGroupIds": "!Import dr-globaldb-cluster-mysql-vpc-security-
group-ids",
        "Engine": "!Import dr-globaldb-cluster-mysql-engine",
        "EngineVersion": "!Import dr-globaldb-cluster-mysql-engine-version",
        "KmsKeyId": "!Import dr-globaldb-cluster-mysql-KmsKeyId",
        "SourceRegion": "!Import dr-globaldb-cluster-mysql-source-region",
        "ClusterRegion": "!Import dr-globaldb-cluster-mysql-cluster-region",
        "BackupRetentionPeriod": "7",
        "MonitoringInterval": "60",
        "StorageEncrypted": "True",
        "EnableIAMDatabaseAuthentication": "True",
        "DeletionProtection": "True",
        "CopyTagsToSnapshot": "True",
        "AutoMinorVersionUpgrade": "True",
        "MonitoringRoleArn": "!Import rds-mysql-instance-RDSMonitoringRole"
    }
}
]
}
]
}

```

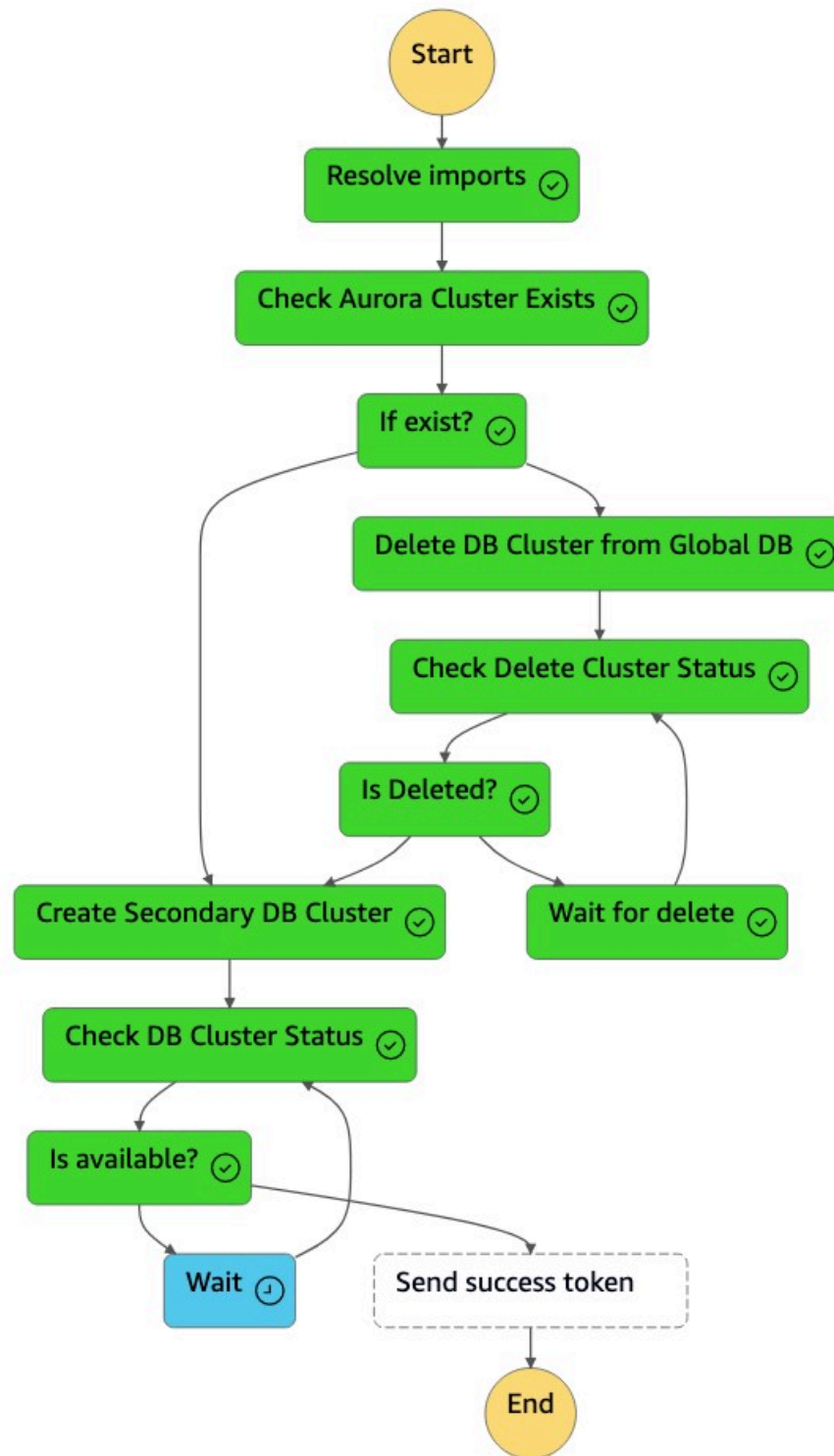
6. The DR Orchestrator FAILBACK state machine reads the resource type as `CreateAuroraSecondaryDBCluster`, and it calls the `dr-orchestrator-stepfunction-create-Aurora-Secondary-cluster` state machine.



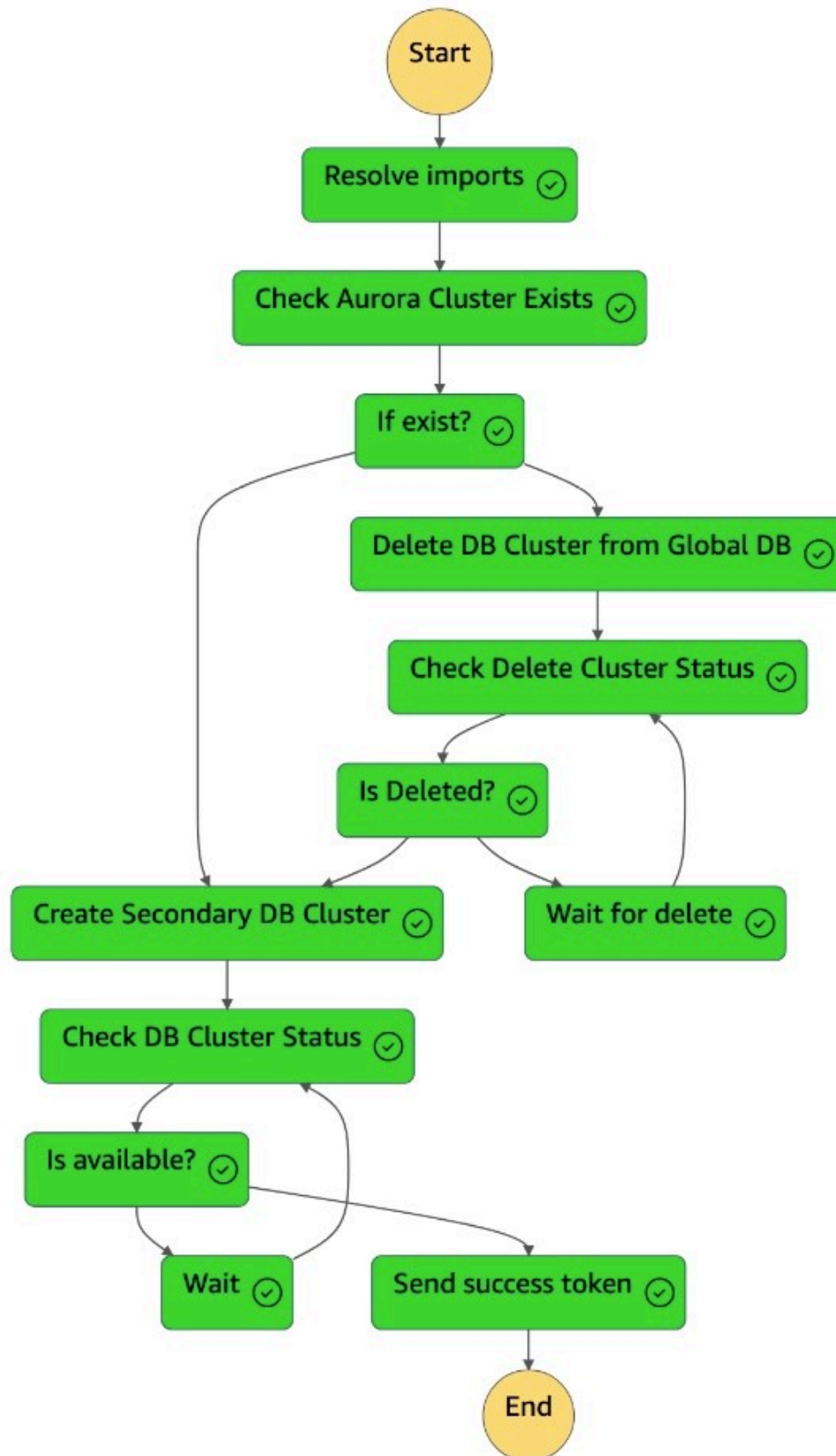
7. The `dr-orchestrator-stepfunction-create-Aurora-Secondary-cluster` state machine deletes the existing cluster (`dbcluster-01`) from the former primary Region (`us-east-1`).



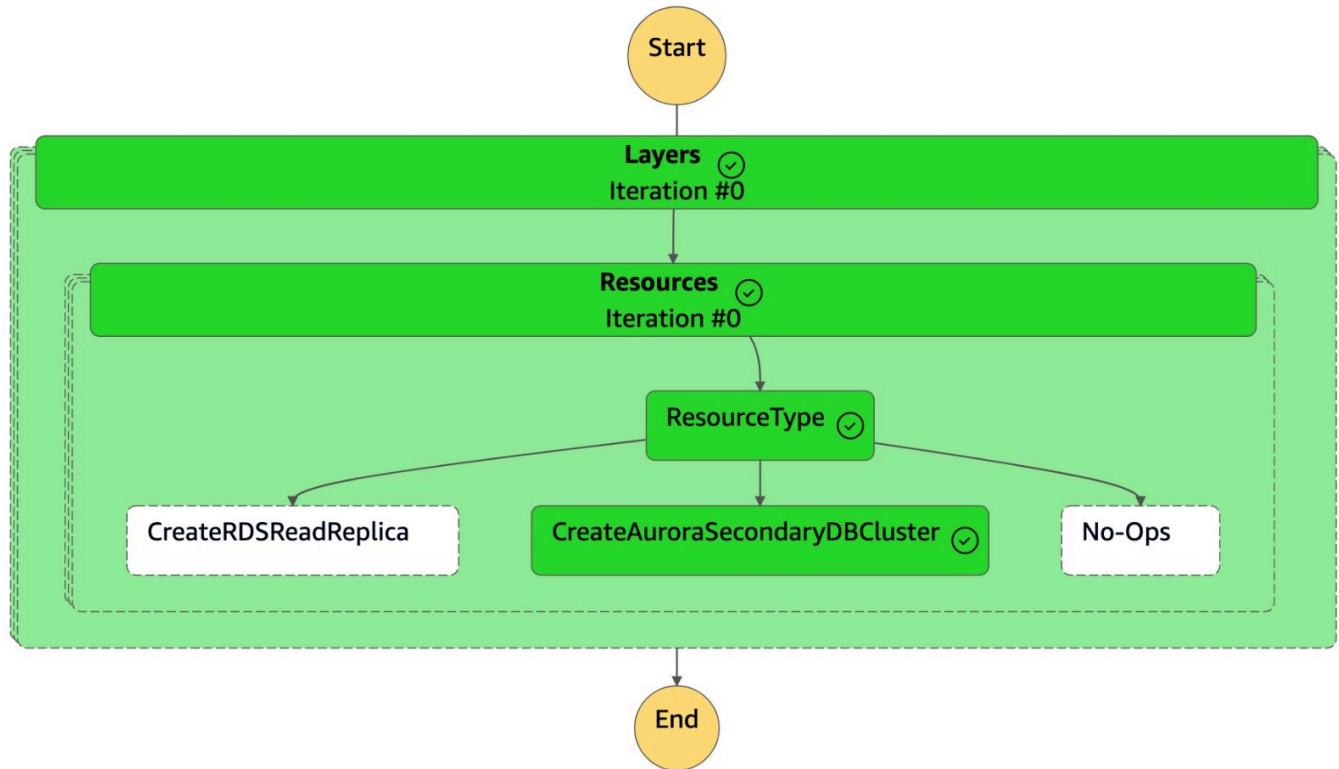
8. After the cluster (dbcluster-01) is deleted, the state machine creates a new cluster (dbcluster-01) along with the DB instance, and it joins the Aurora global database as the secondary cluster to serve read-only workloads.



9. After the secondary cluster is available, the `dr-orchestrator-stepfunction-create-Aurora-Secondary-cluster` state machine is completed, and it sends a success token back to the `DR Orchestrator Failback` state machine.



10.The dr-orchestrator-stepfunction-FAILBACK state machine is completed.



11.You can verify the Aurora global database on the Amazon RDS console.

If you want to relocate the primary DB cluster to us-east-1 then you can follow the steps mentioned in the [Switchover](#) section.

Deploy the solution

After you set up the prerequisites, follow the instructions in the [DR Orchestrator Framework pattern](#). Instructions are included for Amazon RDS Aurora, and Amazon ElastiCache. To test the solution, review the prerequisites and limitations, and deploy example databases on AWS.

Prerequisites

- An active [AWS account](#) to deploy this solution.
- Permissions to create [Amazon Simple Storage Service \(Amazon S3\)](#) buckets and the resources used in the [AWS CloudFormation](#) templates.
- Before you run the DR Orchestrator FAILBACK state machine:
 - Manually [modify the RDS instance](#) or [modify the Amazon Aurora DB Cluster](#), and disable DeletionProtection.
 - Take a [DB snapshot](#) before deleting the Amazon RDS instance or Aurora cluster if you plan to perform root cause analysis (RCA).

Limitations

- Feature availability and support vary across specific versions of each database engine and across AWS Regions. For more information on feature and Region availability for cross-Region replication, see [Cross-Region read replicas](#).
- This solution has been tested using a primary database instance with one read replica. If you want to use more than one read replica, test the solution thoroughly before implementing it in a production environment.
- For Amazon RDS, the solution supports only the following engines: [Amazon RDS for MySQL](#), [Amazon RDS for PostgreSQL](#), and [Amazon RDS for MariaDB](#).

Deploy example databases on AWS

To demonstrate the cross-Region DR solution for databases on AWS, you can deploy [Amazon RDS for MySQL](#), [Amazon Aurora MySQL-Compatible Edition](#), and [Amazon ElastiCache \(Redis OSS\) - Global Datastore](#). Use the instructions in the App-Stack README file of the [aws-cross-region-dr-](#)

[databases](#) GitHub repository. In this example, the primary Region is us-east-1, and the secondary Region is us-west-2.

The application stack uses the AWS CloudFormation [Outputs](#) section to export the parameters that are used by DR Orchestrator Framework to automate the failover and failback of AWS databases.

FAQ

Find answers to questions about DR Orchestrator Framework for failover and failback.

What are the RPO and RTO I can achieve using this approach?

For information about recovery time objective (RTO) and recovery point objective (RPO), see the guide [Disaster recovery strategy for databases on AWS](#).

Is it mandatory to use AWS CloudFormation export variables?

No, you can directly pass the value of the Amazon Aurora global database or the Amazon RDS DB instance directly in the JSON format (for example: - "RDSInstanceIdentifier": "rds-mysql-instance").

Can I use the DR Orchestrator FAILOVER workflow to fail over more than one AWS database?

Yes, you can pass more than one resource in the *input parameter* file to fail over more than one AWS database. The following code example shows failing over an Amazon RDS for MySQL read replica and an Amazon ElastiCache (Redis OSS) global datastore in parallel:

```
{
  "StatePayload": [
    {
      "layer": 1,
      "resources": [
        {
          "resourceType": "PromoteRDSReadReplica",
          "resourceName": "Promote RDS MySQL Read Replica",
          "parameters": {
            "RDSInstanceIdentifier": "!Import rds-mysql-instance-identifier",
            "TargetClusterIdentifier": "!Import rds-mysql-instance-global-arn"
          }
        },
        {
          "resourceType": "FailoverElastiCacheCluster",
          "resourceName": "Failover ElastiCache Cluster",
```

```
        "parameters": {
            "GlobalReplicationGroupId": "!Import demo-redis-cluster-global-replication-
group-id",
            "TargetRegion": "!Import demo-redis-cluster-target-region",
            "TargetReplicationGroupId": "!Import demo-redis-cluster-target-replication-
group-id"
        }
    }
}
```

How can I avoid the InvalidParameterCombination error when I run the DR Orchestrator FAILBACK state machine for Amazon RDS?

The full text of the error is:

```
"errorMessage": "An error occurred (InvalidParameterCombination) when
calling the DeleteDBInstance operation: Cannot delete protected DB
Instance, please disable deletion protection and try again."
```

To avoid the error, [modify the RDS instance](#) by disabling DeletionProtection before you run the DR Orchestrator FAILBACK state machine.

Next steps

You can use DR Orchestrator Framework to add an approval process before initiating the failover or failback activities. For example, while initiating the failover during a DR event, you can add a state machine on top of DR Orchestrator Failover to send an [Amazon Simple Notification Service \(Amazon SNS\)](#) notification through email. As soon as the approval is granted, the failover activity will start.

Application failover can also be integrated with DR Orchestrator Framework to switch the DNS entry and point applications to the new endpoint of the database instance or cluster.

You can use [AWS X-Ray](#) tracing to obtain the failover duration for calculating the RTO. You can build a monitoring dashboard on top of X-Ray.

Resources

- [Replication across AWS Regions using global datastores](#) (Amazon ElastiCache (Redis OSS) documentation)
- [Using failover in an Amazon Aurora global database](#) (Aurora documentation)
- [Automate Amazon Aurora Global Database endpoint management for planned and unplanned failover](#) (blog post)
- [Disaster recovery: 3 failover scenarios for your Amazon Aurora global database with Terraform \(Part 2\)](#) (blog post)
- [Disaster recovery strategy for databases on AWS](#) (AWS Prescriptive Guidance strategy)
- [Automate cross-Region failover and failback by using DR Orchestrator Framework](#) (AWS Prescriptive Guidance pattern)

Document history

The following table describes significant changes to this guide.

Change	Description	Date
Initial publication	—	May 3, 2024